

Rapport technique — Architecture logicielle

Application MiniShop (Jakarta EE, NetBeans, GlassFish)

Document genere automatiquement — Comparaison ancienne vs nouvelle architecture.

1. Contexte

MiniShop est une application e-commerce de demonstration basee sur Jakarta EE : couche presentation (JSP, servlet), logique metier (EJB stateless / stateful), acces aux donnees (DAO, JPA) et base SQL. Le projet a ete restructure pour introduire des **ports** (interfaces) entre la servlet et les EJB, dans une demarche proche de l'architecture hexagonale.

2. Limites de l'ancienne architecture

L'ancienne version suivait une organisation en couches classique ou la servlet connaissait directement les **implementations** nommees (MiniShopLess, MiniShopFull, OrderService). Les principales limites sont :

- **Couplage fort presentation / metier** : la servlet importait les classes EJB concretes ; tout renommage ou remplacement d'un bean impactait la couche web.
- **Fuite d'information vers la presentation** : la servlet dependait des details techniques des beans (d'ou la mention « details connus par la servlet » sur le schema).
- **Commande liee au type concret du panier** : OrderService utilisait MiniShopFull comme type de parametre au lieu d'une abstraction « panier ».
- **Evolutive et tests** : plus difficile de substituer une implementation ou de tester sans monter l'integralite des EJB.
- **Manque de contrat explicite** : pas d'interfaces nommees pour catalogue, panier et validation de commande.

Figure 1 — Ancienne architecture (couches directes)

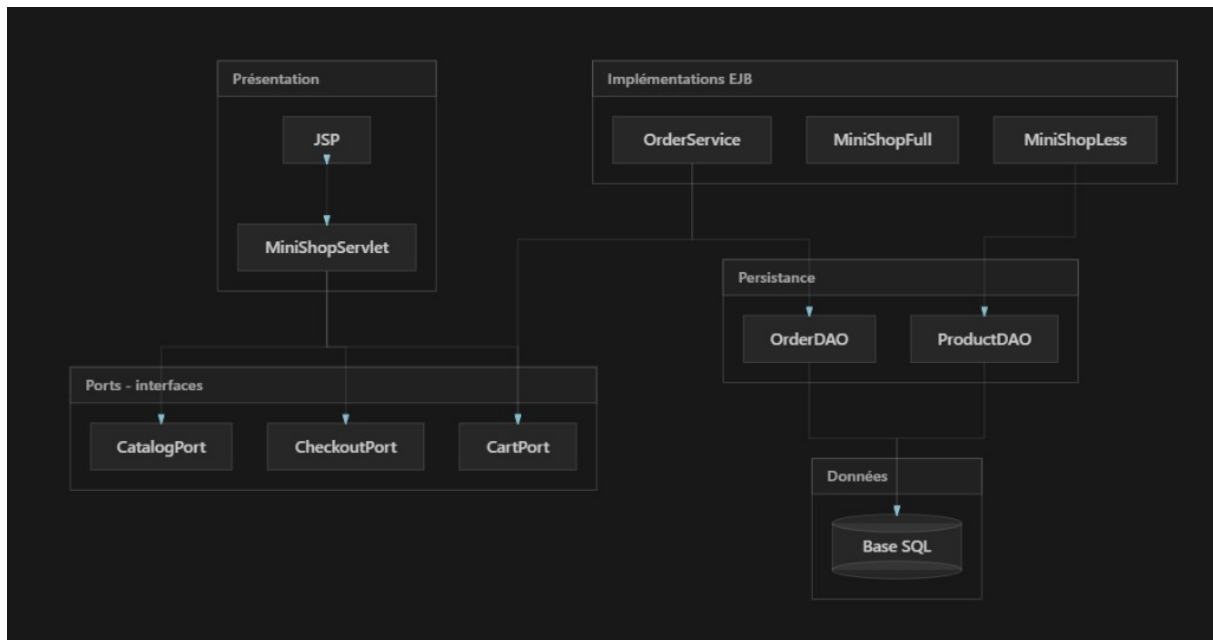


3. Points forts de la nouvelle architecture

La nouvelle version introduit une couche **Ports — interfaces** (CatalogPort, CartPort, CheckoutPort) entre la présentation et les EJB. Les beans existants **implementent** ces interfaces ; la servlet n'injecte plus que les ports.

- **Decouplage** : la servlet ne depend plus des noms de classes EJB, seulement des contrats.
- **Abstraction du panier** : CheckoutPort.confirmOrder(..., CartPort) exprime le metier sans lier a MiniShopFull.
- **Clarte des responsabilites** : trois ports alignes sur les cas d'usage (catalogue, panier, commande).
- **Alignement avec les DAO existants** : ProductDAO et OrderDAO restent les adaptateurs vers la persistance.
- **Verifiabilite** : les regles ArchUnit peuvent imposer que la presentation n'importe pas le package EJB.

Figure 2 — Nouvelle architecture (ports et implementations EJB)



4. Pourquoi ce choix architectural ?

Ce choix equilibre **theorie** (hexagonale, inversion des dependances) et **pratique** de projet : pas de microservices ni de reecriture complete, mais des interfaces explicites et une servlet qui ne connait plus les implementations. C'est adapte a un enseignement Jakarta EE et facilite la maintenance et l'evolution future (par exemple exposition REST sur les memes ports).

5. Comparaison synthetique

Critere	Ancienne architecture	Nouvelle architecture
Dependances de la servlet	Classes EJB concretes	Interfaces CatalogPort, CartPort, CheckoutPort
Couplage	Fort (noms de beans dans la presentation)	Reduit (contrats stables)
Commande / panier	OrderService(MiniShopFull)	CheckoutPort + CartPort
DAO / base	Inchange	Inchange
Tests & evolution	Plus sensibles aux refactorings	Interfaces plus faciles a mocker / substituer

Conclusion. L'introduction des ports ne change pas le comportement fonctionnel ni la stack technique (GlassFish, JPA), mais ameliore la structure du code et prepare de futures extensions avec moins de friction.